CONTENT BASED STOCK MARKET PREDICTION SYSTEM USING MACHINE LEARNING

Major Project Report

Major Project Report by :

DIBAKAR DAS
University Registration Number : 10800123080

DIPIKA MAJI
University Registration Number : 10800123083

INDRAJIT KHAN
University Registration Number : 10800123092

CHANCHAL KUMARI
University Registration Number : 10800123067

ANUSHKA MISHRA
University Registration Number : 10800123035

Under the Guidance of
Dr. ARNAB CHAKRABORTY
Assistant Professor

INTRODUCTION

It’s an app where you can type in a stock ticker (like AAPL, TSLA) → it fetches real-time data → shows stock history → and predicts if the stock will go up or down.

✦ **Frontend** → React + Next.js → shows charts, predictions, and stock data.

✦ **Backend** → Django REST API → handles stock data, runs ML models, and serves predictions.

✦ **ML Models** → Trained on stock market data (Yahoo Finance API) using **Random Forest & Ensemble models**.

✦ **Caching** → Redis (to make repeated stock queries fast).

✦ **Deployment** (Future Aspect) → Backend on **Heroku**, frontend on **Vercel**.

StockVibePredictor - Theoretical Overview

StockVibePredictor is a **full-stack web application** that uses **machine learning** to predict stock market movements. It connects **real-time stock data**, processes it with trained models, and then shows predictions in an interactive dashboard.

It's basically where **AI meets finance** — with a clear **frontend (UI), backend (logic + API), and ML (intelligence)** layer.

MAIN COMPONENTS

1. Frontend (React + Next.js + Chart.js)

✦ The **user interface** where you type in a stock ticker (like AAPL or TSLA).
✦ Shows **historical price charts**, **technical indicators**, and **predictions**.
✦ Mobile-first and interactive → works well on desktop and phone.
✦ Uses **Chart.js** to visualize price trends and ML predictions.

2. Backend (Django + REST API)

✦ Acts as the **brain** behind the UI.
✦ Accepts requests from frontend → processes them → sends back predictions/data.
✦ Exposes APIs like :
- ✧ Health check (system status)
- ✧ Stock prediction
- ✧ Stock data retrieval
- ✧ Model listings

✦ Manages the business logic: connecting ML models, stock data, and Redis.

3. Machine Learning Models (Scikit-learn, Random Forest, Ensemble)

✦ Trained on **historical stock market data** (via Yahoo Finance API).

✦ Uses **technical indicators** such as:

- ✧ RSI (Relative Strength Index)
- ✧ Moving Averages
- ✧ Other time-based features.

✦ Predicts whether a stock will go **Up or Down** for different timeframes (1 day, 1 week, 1 month, etc.).

✦ Models are versioned and regularly retrained to stay up to date.

4. Data Source (Yahoo Finance API)

✦ Provides **real-time stock prices, volume, and history**.

✦ Ensures predictions are made on **latest available data**.

✦ Integrated directly into both training and live predictions.

5. Redis (Caching Layer)

✦ Stores frequently accessed stock data and prediction results.
✦ Makes the system **faster** and reduces repeated API calls.
✦ Critical for real-time performance.

6. Database (SQLite/PostgreSQL)

✦ Keeps records of data, users, and possibly stored models.
✦ Ensures persistence across sessions and deployments.

7. Deployment (Heroku + Vercel) [OPTIONAL]

✦ **Backend** (Django + API) runs on Heroku.
✦ **Frontend** (React UI) runs on Vercel.
✦ Together, they make the system accessible publicly.

KEY FEATURES

1. **Live Market Data (Yahoo Finance API via(finance)**

✦ **What it is** :
The project connects to **Yahoo Finance** using the Python's yFinance library **real-time and historical stock prices**.

✦ **Why it matters** :
Predictions are only useful if the underlying data is accurate and up to date. By pulling live prices, volume, and historical charts, the app ensures predictions reflect **current market conditions**.

✦ **How it works (theory)** :
✦ User requests data for a stock (e.g., AAPL).
✦ Backend calls the Yahoo Finance API.
✦ Data is cached in Redis for faster reuse.
✦ Data feeds into both the frontend charts and the ML prediction pipeline.

2. **ML-Powered Predictions (Trained Models)**

✦ **What it is** :
The app uses **machine learning models** (Random Forest, ensemble models) trained on historical stock data to predict whether a stock will **go up or down** in the next time period (1 day, 1 week, etc.).

✦ **Why it matters** :
Instead of raw numbers, users get **actionable insights** → a simplified "trend direction" prediction that helps in analysis or research.

✦ **How it works (theory)** :
✧ Historical stock data is collected (past prices, volume, technical indicators like RSI, Moving Averages).
✧ ML model learns patterns (for example: when RSI is low, stocks often rebound).
✧ At runtime, the trained model is applied to new live data → outputs a prediction (Up/Down) with accuracy metrics.

3. Charts → Interactive Visualizations with Predictions

✦ **What it is** :
The frontend uses **Chart.js** to display **time-series stock charts** (candlestick/line graphs) with overlays for predictions.

✦ **Why it matters** :
Visual representation helps users **see both history and prediction trends clearly**, making it easier to analyze whether predictions align with real movements.

✦ **How it works (theory)** :
✧ Frontend receives processed data (historical prices + prediction results).
✧ Charts are rendered with time axes, prices, and optionally model prediction markers.
✧ Users can hover over points to see detailed values.
✧ The interface is responsive, so charts adapt to mobile or desktop seamlessly.

4. **REST API → Fetch Predictions Programmatically**

✦ **What it is** :
The backend exposes an **API (Application Programming Interface)** so predictions and data can be accessed outside the UI.

✦ **Why it matters**: Developers or other apps can connect to StockVibePredictor directly to :
✧ Get predictions for specific stocks.
✧ Retrieve stock history or available models.
✧ Integrate predictions into their own trading dashboards or bots.

✦ **How it works (theory)** :
✧ Backend provides endpoints like :

/api/predict/	→ get predictions for a stock.
/api/stock/{ticker}/	→ get stock history.
/api/models/list/	→ see all available models.

✧ Users send requests → backend responds with JSON-formatted results.

5. **Mobile-First Responsive UI**

✦ **What it is** :
The frontend is designed with **responsive layouts** so it works smoothly on both desktops and smartphones.

✦ **Why it matters**:
Many users check stocks on the go. A mobile-first design ensures predictions and charts are usable even on smaller screens.

✦ **How it works (theory)**:
✧ Built with **React + Next.js** → provides dynamic UI.
✧ Uses CSS frameworks to adapt to screen sizes (scalable charts, collapsible menus).
✧ The same app works on laptops, tablets, and phones without separate versions.

TECH STACK OVERVIEW

Machine Learning :
✦ **Scikit-learn** → Random Forest for predictions.
✦ **Numpy, Pandas** → data handling, ML support.

Backend :
✦ **Django + Django REST Framework** → APIs.
✦ **Redis** → cache stock data & predictions.

Frontend :
✦ **React + Next.js** → UI.
✦ **Chart.js** → visual stock charts.

Storage :
✦ **SQLite/PostgreSQL** → for persistent data.
✦ **Models stored as .pkl** files.

How Everything Connects (Workflow)

✦ **User Interaction**: You enter a stock ticker (like AAPL).
✦ **Frontend Request**: The frontend sends this request to the backend API.
✦ **Backend Processing**:
✦ Checks Redis cache for existing data.
✦ If missing, fetches fresh data from Yahoo Finance API.
✦ Passes the data to ML models for prediction.
✦ **ML Prediction**: Models analyze technical indicators → predict movement.
✦ **Backend Response**: Sends prediction + data back to frontend.
✦ **Visualization**: Frontend shows stock chart + prediction + confidence.

CONCLUSION

The app provides a simple yet powerful way to analyze and predict stock movements. By allowing users to enter a stock ticker, it automatically fetches live market data, displays historical price trends in interactive charts, and uses machine learning models to forecast whether the stock is likely to move up or down. This combination of **real-time data, visual insights, and predictive intelligence** makes the tool practical for both beginners and experienced investors. Overall, it bridges the gap between raw financial data and actionable insights, offering a responsive and accessible platform for stock market research.

THANK YOU